



Faculty of Engineering and Applied Science

SOFE 4U: Machine Learning

NBA Line Prediction: Report

Due Date: March 19th, 2025

Date submitted: March 19th 2025

Group 1

Juliano Falotico (100658311)

Ontario Tech University

Oshawa, Ontario

juliano.falotico@ontariotechu.net

Jordan Frost Hagedorn (100828122)

Ontario Tech University

Oshawa, Ontario

jordan.hagedorn@ontariotechu.net

Trent Jordan(100831853)

Ontario Tech University

Oshawa, Ontario

trent.jordan@ontariotechu.net

Introduction

For this project, the goal was to create a machine learning model that can predict the fifth player for the home team of an NBA lineup. Training data was provided which included partial lineup data, game statistics and other metrics. Certain data from the sets was required to be ignored and only specific columns were allowed. By using the allowed data columns, preprocessing the data, then choosing and adjusting our model our team was able to achieve some interesting results.

Data Preprocessing

To ensure that our model can accurately predict the newest player on an NBA lineup, our group created some preprocessing steps to formulate new data that can be used to train the models that predict the fifth player. The first step of this process was to make sure each team's name was consistent throughout the time frame of data given to us as some basketball teams such as the New Orleans Pelicans were originally called the New Orleans Hornets. To achieve this, the data was combed through to find any name changes throughout 2006-2016. If a name was changed, then the training data was searched and the correct name swapped. The next step was to create some new rosters to train on from the training data given to us. This was done by only using rosters that had an outcome of "1" (meaning the home roster did better than the away roster) and then creating five new rosters where each home player gets removed from one of the rosters. By doing this our group is able to quintuple the number of rosters to train on. When quintupling the amount of data we are training on, we increase the accuracy of the model. It is worth noting that the risk of overfitting would also increase by increasing the amount of data the model is trained on, but by using a Random Forest Classification model our group avoids this issue. The last step our group took was to sort these newly created rosters into csv files depending on which player was missing from the roster (a csv file for "home_0" missing, "home_1" missing, and ext). The decision to sort and separate the rosters by which the player was missing was made for two reasons. The first reason being that our group noticed that there was a correlation between a player's position and the "home" column they appear in. Meaning that certain players are more likely to appear in one "home" column than the other. The second reason was that this would drastically reduce the search space of players that could be chosen which should make the prediction more accurate. It should also be mentioned that the test data undergoes the same sorting process as the training data and before the test data is sent to the model to be trained, it is appended to the bottom of the training data where both groups can be label encoded at the same time. By label encoding the training data and test data at the same time our group prevents any error that might occur from not encoding values that only appear in the test set. The group also ended up calculating some player quality scores based off of NBA fantasy scoring but was unable to use them.

Model That was Chosen

The model that was chosen was a Random Forest Classification model. Our group chose this model because it is a high accuracy model that can scale well with the large amount of data our group is dealing with while also being resistant to overfitting. This kind of classification is also good at handling wide varieties of data, in our case the number of unique players would be the large variety.

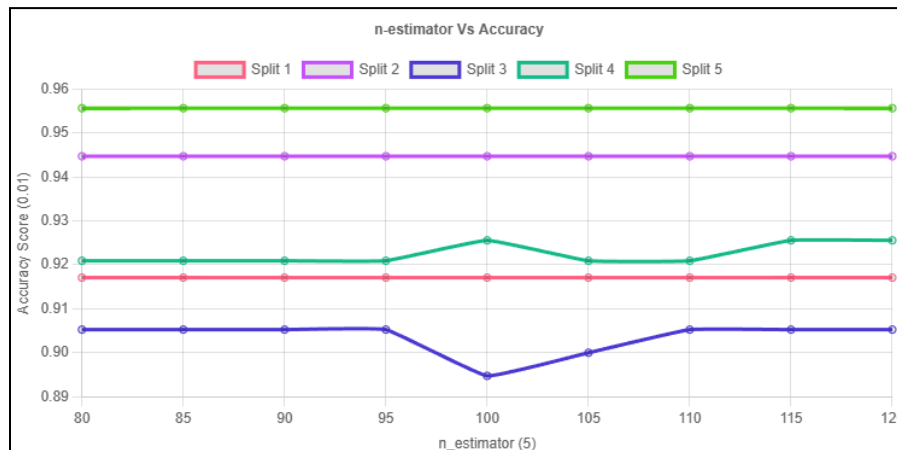
Training, Testing, and Analysis

As mentioned above in the data preprocessing section, our group split the data into five different groups depending on the column of the player that was removed. With this our group also trained five different models to ensure a smaller search space and thus a higher accuracy. The group then tested the models on the five different test sets that were created at the same time as the training data. The results are shown in the table below.

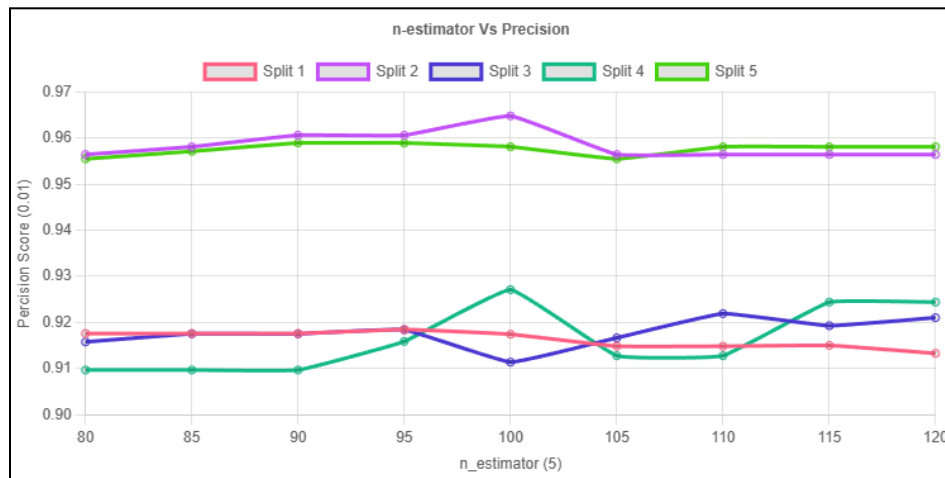
Column Removed	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
home_0	91.71	91.85	91.71	91.38
home_1	94.47	95.64	94.47	94.38
home_2	90.53	91.84	90.53	90.51
home_3	92.09	91.59	92.09	91.22
home_4	95.57	95.80	95.57	95.46

In an attempt to optimize these results, parameters of the model, such as n_estimator amount, were adjusted to see the varying effects it would have on the performance metrics in order to obtain the optimized results above

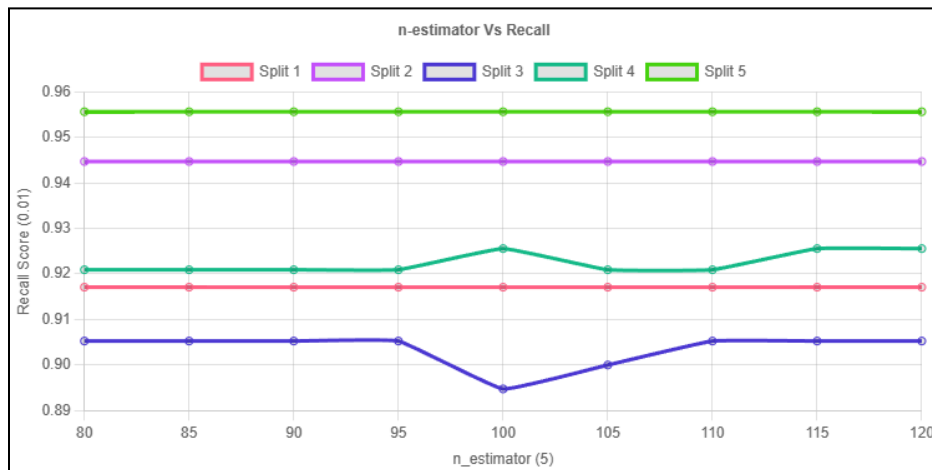
Changes to Accuracy When Changing the Number of n_estimator's



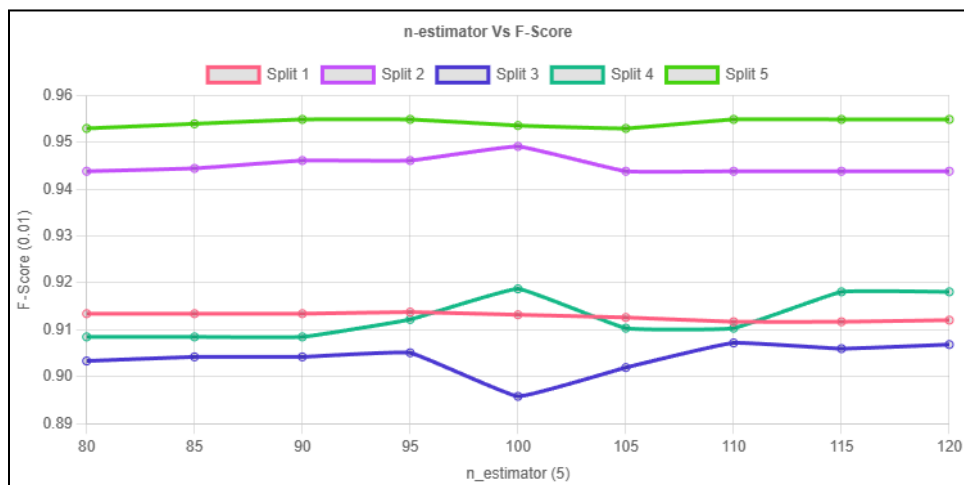
Changes to Precision When Changing the Number of n_estimator's



Changes to Recall When Changing the Number of n_estimator's



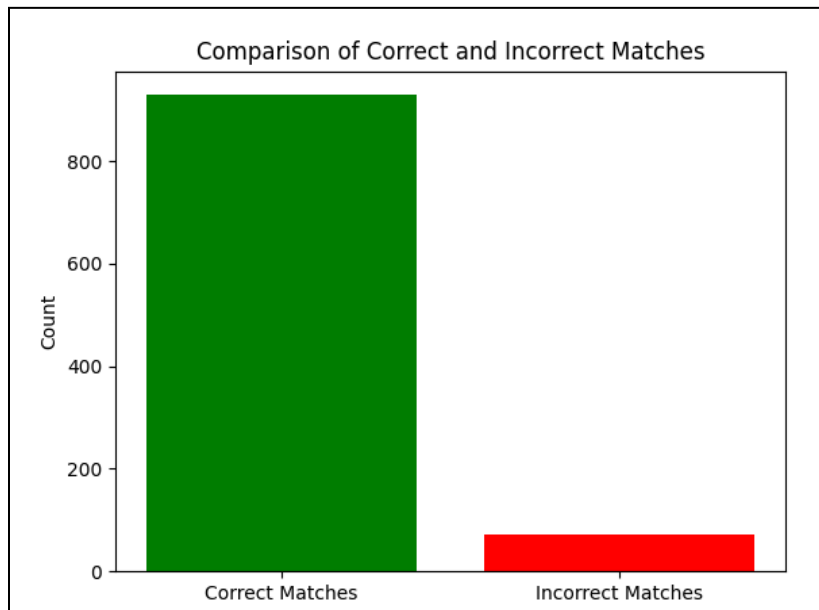
Changes to F-Score When Changing the Number of n_estimator's



The results of changing the models parameters show that increasing the amount of n-estimators has miniscule effects on the performance of the model. Nonetheless an optimal value was chosen after these results were analyzed.

We can also count how many players are correctly predicted in total and for each season, in the table below, the number of correct predictions made for a season can be seen. Note that each season has 100 different test cases.

Total Correct Matches Vs Incorrect Matches



Total Correct Predictions for Each Season

Season	Correct Predictions
2007	100
2008	100
2009	100
2010	100
2011	100
2012	100
2013	100
2014	100
2015	100

2016	29
------	----

We can extrapolate the results and further evaluate the model by player column on the 2016 season since that was the only season the models got wrong. Because the only season that gets predictions wrong is 2016, we can say that if the season was 2016 and the player was missing in the home_0, home_1, home_2, home_3, and home_4 columns we would have a 17.1%, 44.7%, 5.3%, 20.9%, and 55.7% chance respectively for predicting the correct player. This could imply that lineups stay similar to previous seasons when home_1 and home_4 players are missing as these are the most correctly predicted columns. This could also imply that there may be some sort of bias for the testing lineups when these players are removed as it could just be coincidence that all of the players have been seen before in the training data. Therefore if any of the players were removed from these correctly predicted lineups and handed to their receptive model it is possible that the model would've been able to make a correct prediction as a similar lineup may have already been seen.

Conclusion

In conclusion, the model is really good at predicting lineups in seasons that it has data on and really bad at predicting lineups in seasons it had no data on. This can be attributed to the fact that there are a lot of newer players that get played every season and without data on these newer players it is much harder to make a correct prediction. Without the proper data it becomes almost impossible to predict the player that should be on the roster. This missing data could be supplemented with data from each player's scores from the NCAA since most new players that join the NBA are from the NCAA. There could also be bias within the test data but our group has yet to specifically analyze the test data to find such bias.